

linux-image-provisioning

Linux cross-build image provisioning

Sub-package: pkg/crossbuild Backend covered:
LinuxContainerBackend Forensic anchor: iter-54 of Yole,
2026-05-13

What you are building

The pkg/crossbuild package needs a Linux container image with JDK 17 + Gradle pre-installed. The image is the runtime that executes Gradle inside an isolated Linux userspace, producing native Linux artifacts (.deb, .rpm, jpackage runtime images) on any host that has rootless podman or docker available.

Use this image when:

- The host is macOS or Windows + you need a Linux .deb.
- The dedicated Linux build host (.env: LINUX_BUILD_HOST) is unreachable due to a network change, VPN switch, or host outage.
- The dedicated host's system JDK lacks jmods (see ticket #linux-build-host-jdk-jmods-bootstrap in Yole's KNOWN_DEFECTS).

Until this image exists on a given host, pkg/crossbuild's LinuxContainerBackend.Build() returns an actionable error pointing at this document + the #crossbuild-linux-image-provisioning skip-OK ticket.

Provisioning steps (any host, rootless podman)

`cd Submodules/Containers/pkg/crossbuild`

```

# 1. Build the image FOR THE TARGET ARCHITECTURE you intend to
    ship.
#    Build both if you ship to both x86_64 + arm64 Linux users.
podman build \
    --platform linux/amd64 \
    -t ghcr.io/vasic-digital/crossbuild-linux:jdk17-amd64 \
    -f linux_container.Containerfile .

podman build \
    --platform linux/arm64 \
    -t ghcr.io/vasic-digital/crossbuild-linux:jdk17-arm64 \
    -f linux_container.Containerfile .

# 2. Smoke check (per arch).
podman run --
    rm ghcr.io/vasic-digital/crossbuild-linux:jdk17-amd64
    gradle --version

# 3. (Optional) Push to the registry for the other hosts.
podman push ghcr.io/vasic-digital/crossbuild-linux:jdk17-amd64
podman push ghcr.io/vasic-digital/crossbuild-linux:jdk17-arm64

```

On Apple Silicon hosts: the `--platform linux/amd64` variant runs under QEMU translation (slower); the `linux/arm64` variant runs natively (fast). Choose based on which artifact your users actually need.

Verifying integration with pkg/crossbuild

```

cd Submodules/Containers
go test ./pkg/crossbuild/... -race -count=1

```

Expected: PASS (uses injected fakes, doesn't need the image).

Real-stack Challenge

challenges/crossbuild_linux_deb_challenge.sh:

Expected on a host WITH the image provisioned:

```

OK: produced Yole-Desktop-linux-x64-1.0.1-Release-0.0.0.1.1.deb
(size N bytes)
PASS: crossbuild_linux_deb_challenge

```

Expected when the image is NOT yet provisioned:

```

SKIP-OK: #crossbuild-linux-image-provisioning - ghcr.io/vasic-
digital/crossbuild-linux:jdk17-amd64 not on host

```

A PASS without an actual non-zero .deb is a bluff under CONST-035 §11.4 + the operator's iter-54 anti-bluff mandate. The Challenge enforces this via `[[ -s "$OUTPUT_DEB" ]] post-condition.`

Decoupling note

This package is FULLY decoupled from any specific build host (LINUX_BUILD_HOST env var or otherwise). The Backend works on any host with rootless podman/docker; the operator's choice of which host to run it on is configuration, not code. This is the operator mandate from Yole iter-54 (2026-05-13): *"Make sure nezha.local is in configuration file, not hardcoded in the project anywhere!"*